

Who wants to go and
build a system in **Elixir**?



Dave Thomas

@pragdave



Following

Reading patterns books is like watching drug ads on TV. By the end, you're convinced you have all the symptoms and need all the remedies.

RETWEETS

93

LIKES

128



11:47 PM - 25 Oct 2016

REWRITE ALL THE APPS



YOU CAN'T



Can't go out and
rewrite it all

You shouldn't

Elixir, your Monolith and You

(alpha-1)

Tobias Pfeiffer

@PragTob

pragtob.info



L I E F E R Y

Your **Monolith** and **You**



Your Monolith now?

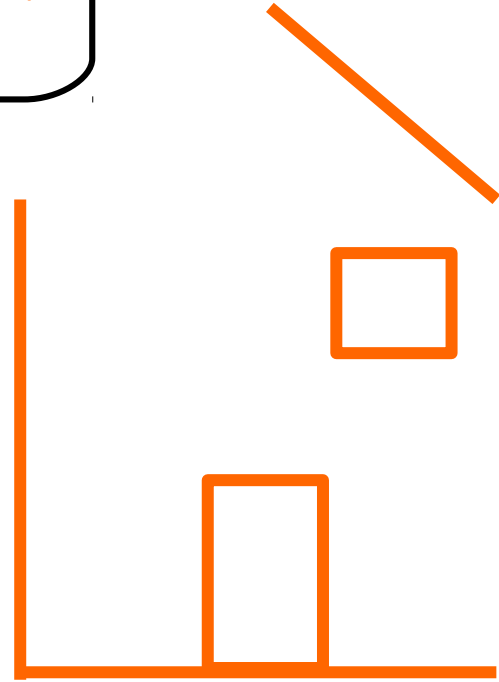
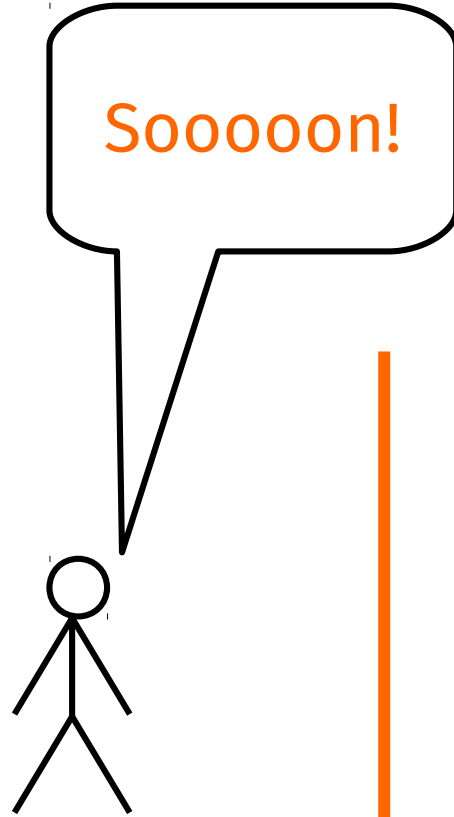
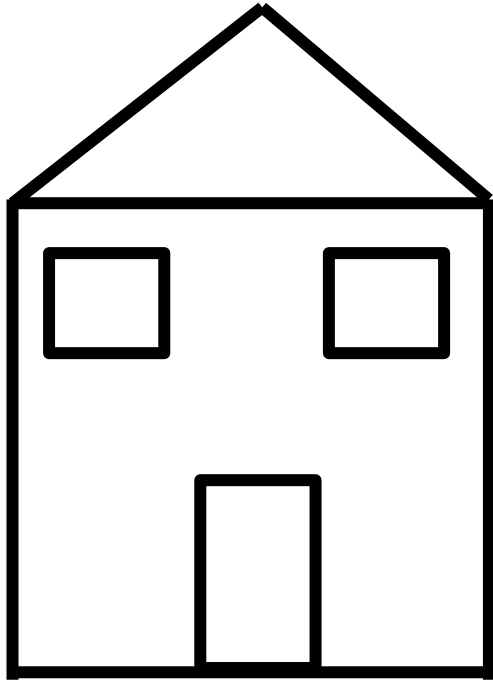


Your Monolith?



Tear it **all down** and
rewrite it





*“We rewrote it all in
Language x, now it’s 10
times faster and only
50% of the code!”*

Someone

Often implies...

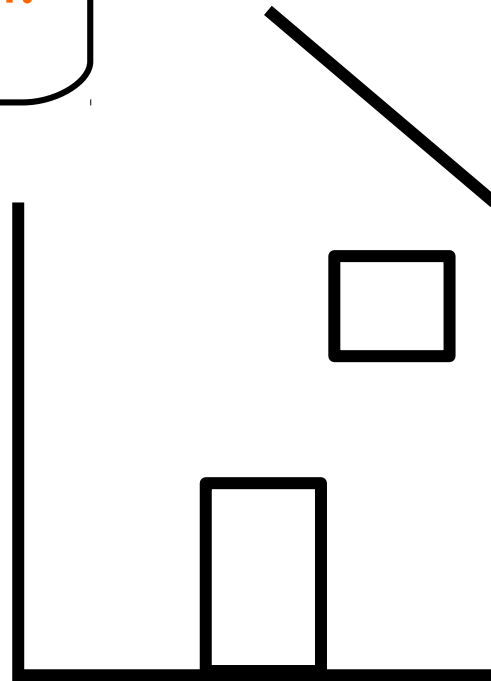
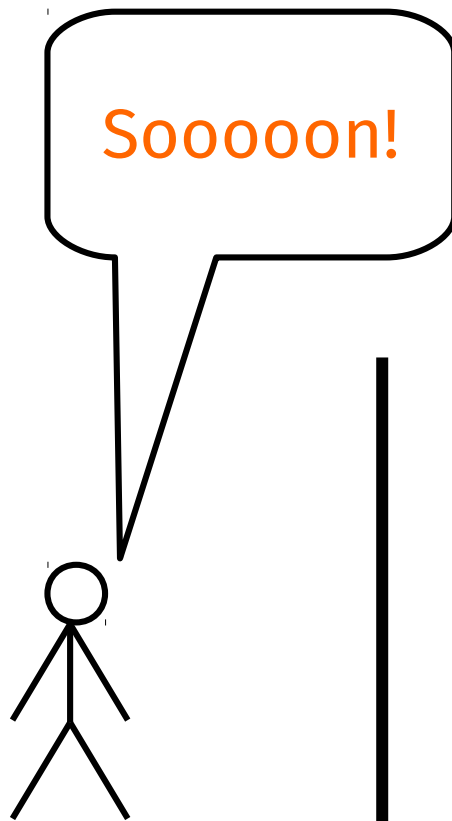
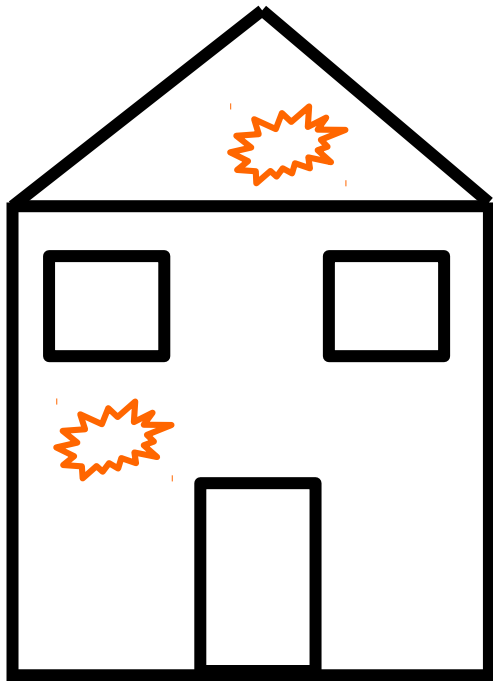
“Language x, is 10 times faster and requires 50% of the code of language Y!”

Someone

What nobody tells you...

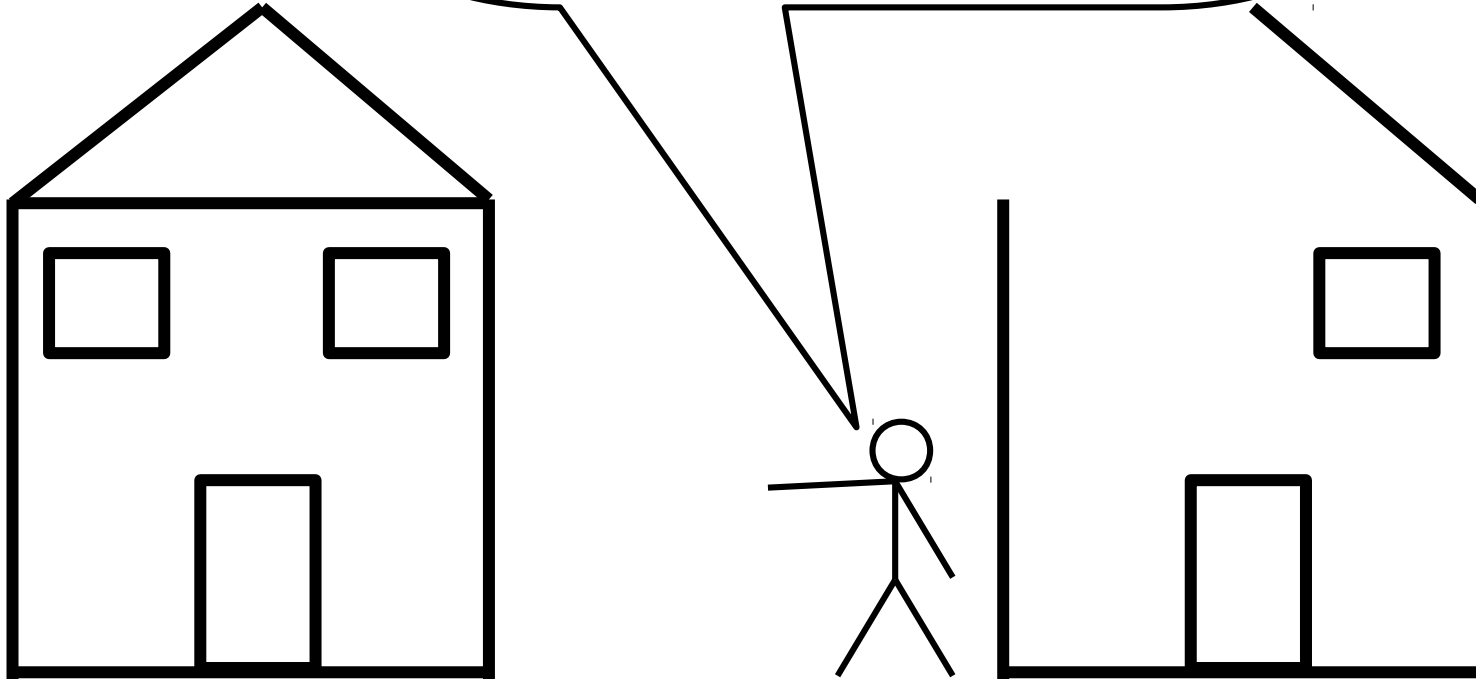


Business Value?



Replace it **step by step**

We have a **bedroom**,
for the bathroom please
check out **legacy**!



Terraform allows you to
incrementally transform an
older API into one powered by
Phoenix - *one endpoint at a
time.*


```
defmodule MyApp.Terraformers.Foo do
  alias MyApp.Clients.Foo
  use Plug.Router

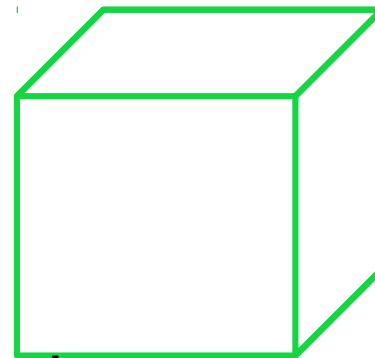
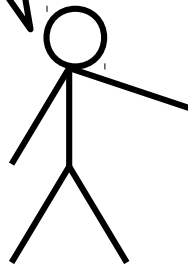
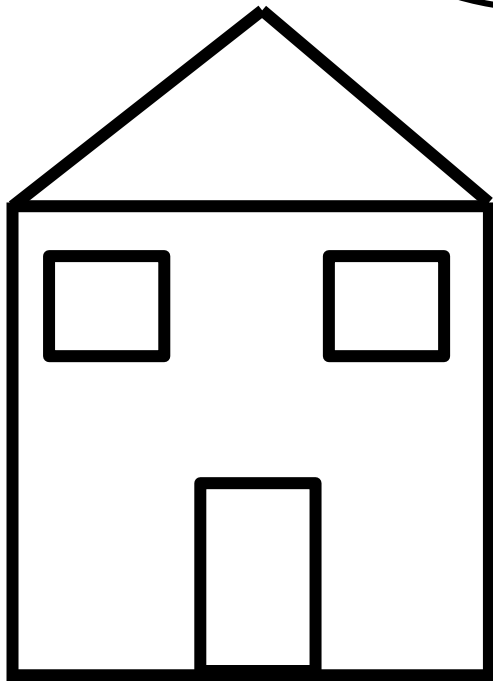
  plug :match
  plug :dispatch

  get "/v1/hello-world", do: send_resp(conn, 200, "Hi")

  get _ do
    res = Foo.get!(conn)
    send_response({:ok, conn, res})
  end
end
```

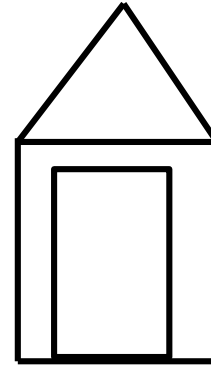
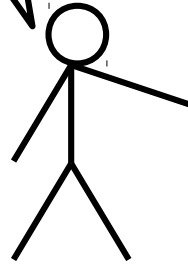
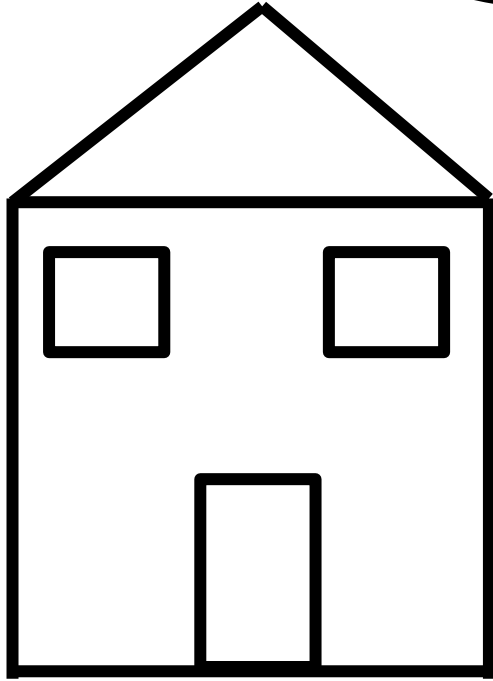
Replace a **critical**
component

Check out our sweet **new**
heating system!



Write a **new** component

Check it out we built a
brand new high tech
entertainment facility!



Obvious in microservices

Write a **new** component

*“Is the **technology** I’m
choosing the **right fit** for
the **problem** I’m trying to
solve?”*

Everyone, hopefully

Reasonably **separate and**
limited problem

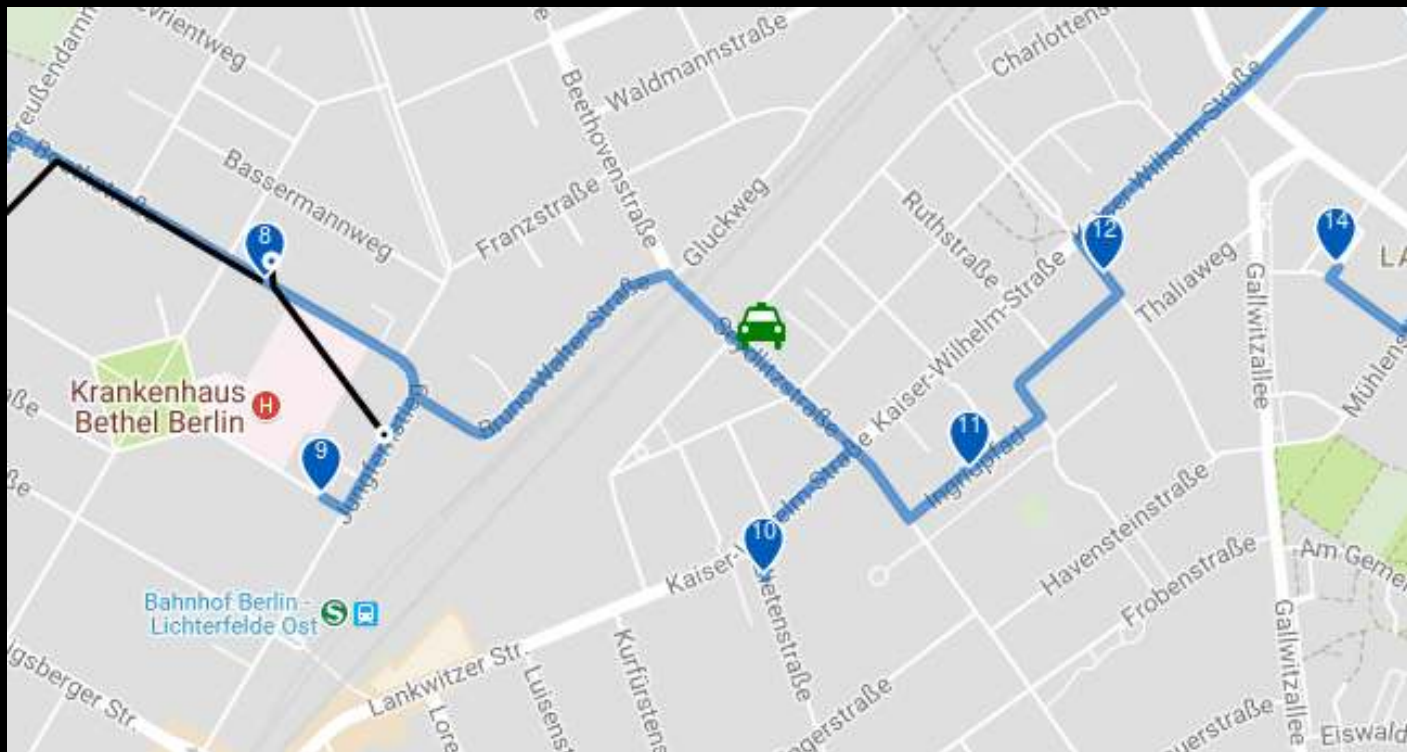
Requirement that poses a
problem for the current
stack

Spark of Enthusiasm

Courier Tracking



Courier Tracking



Courier Tracking



Courier Tracking

Websockets



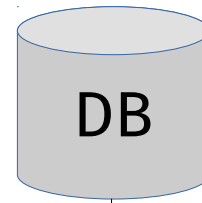
Courier Tracking

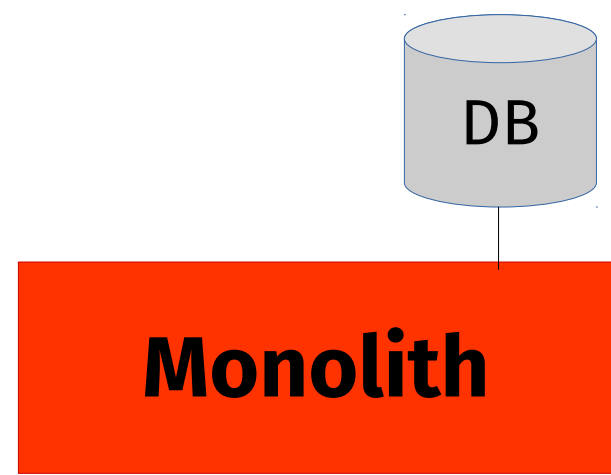
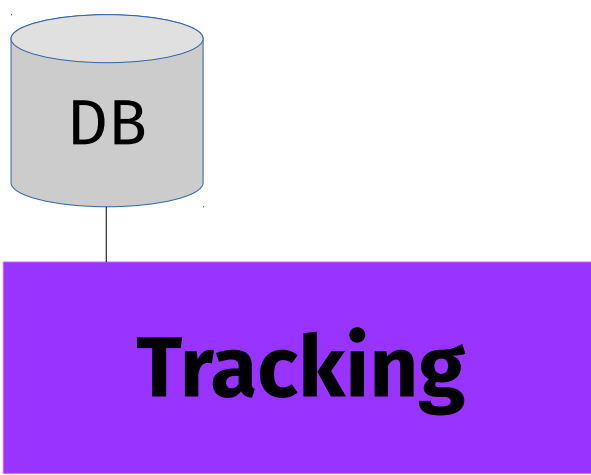
Websockets

Basic Knowledge

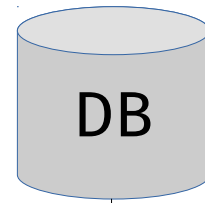
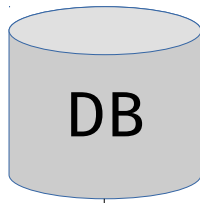


How to make them **talk**?





Connect them?



JSON

Web

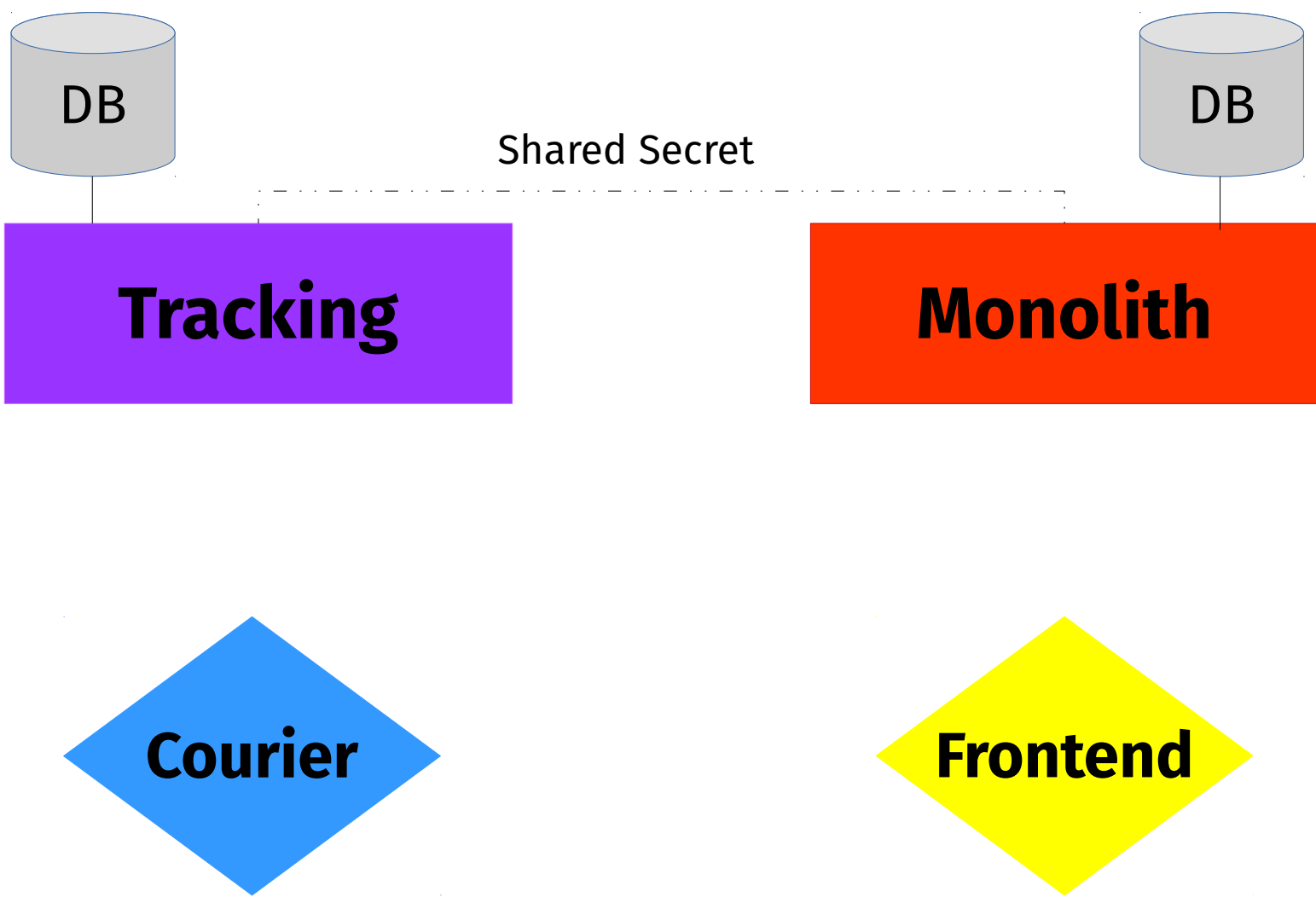
Token

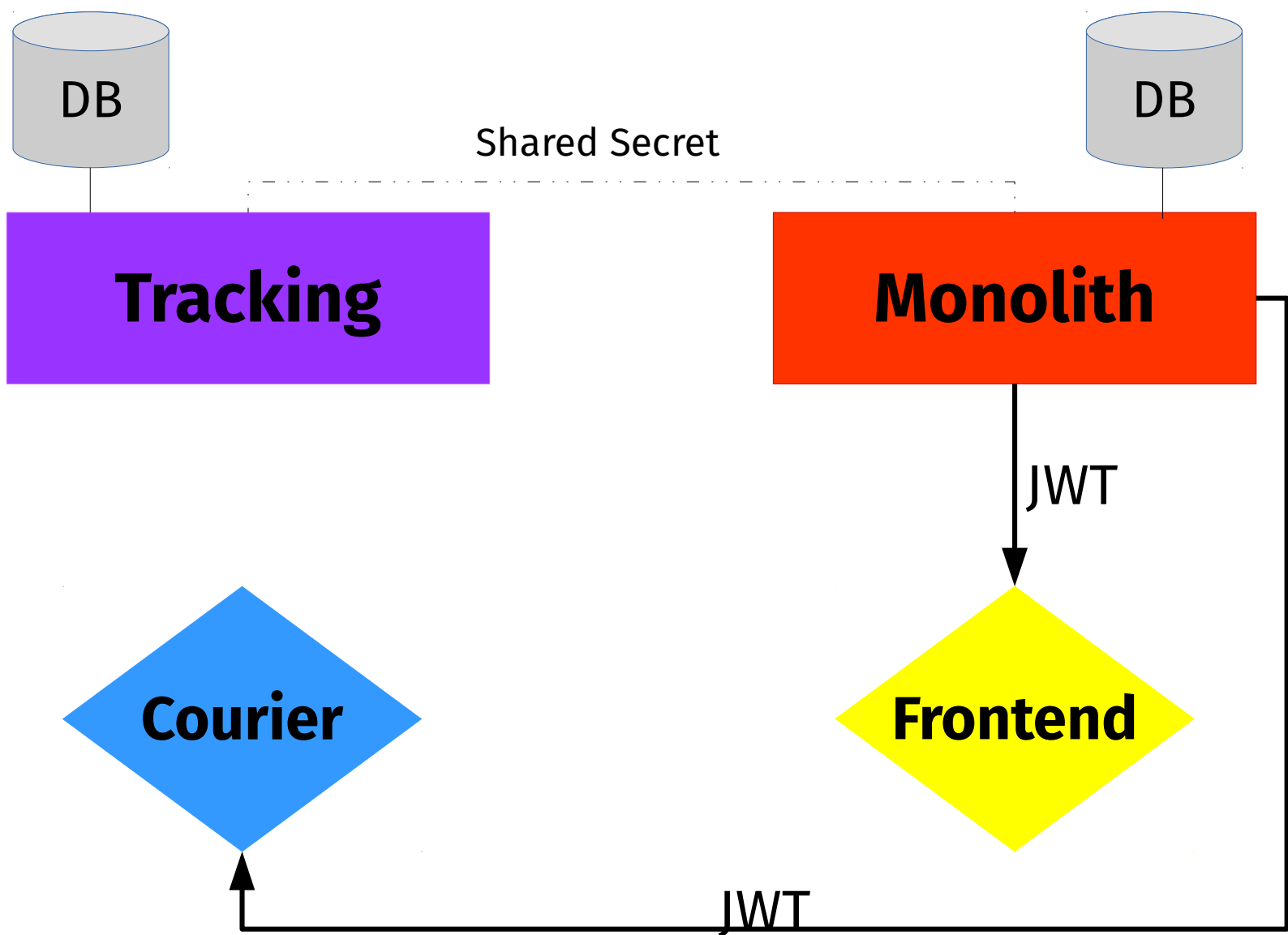
Header

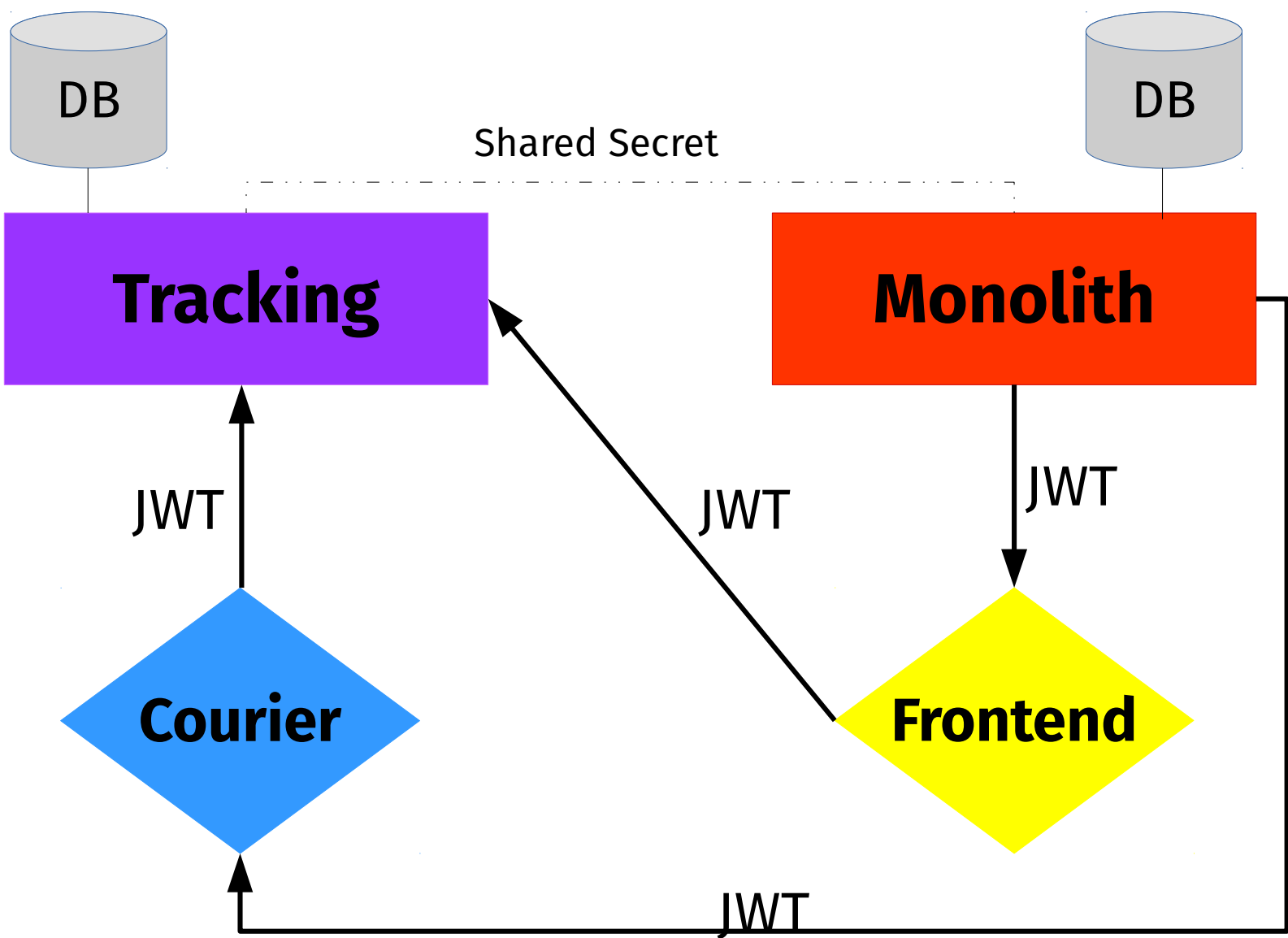
Payload

Signature

```
{  
  "role": "admin",  
  "courier_ids": [1337, 55],  
  "id": 42,  
  "exp": 1520469475  
}
```







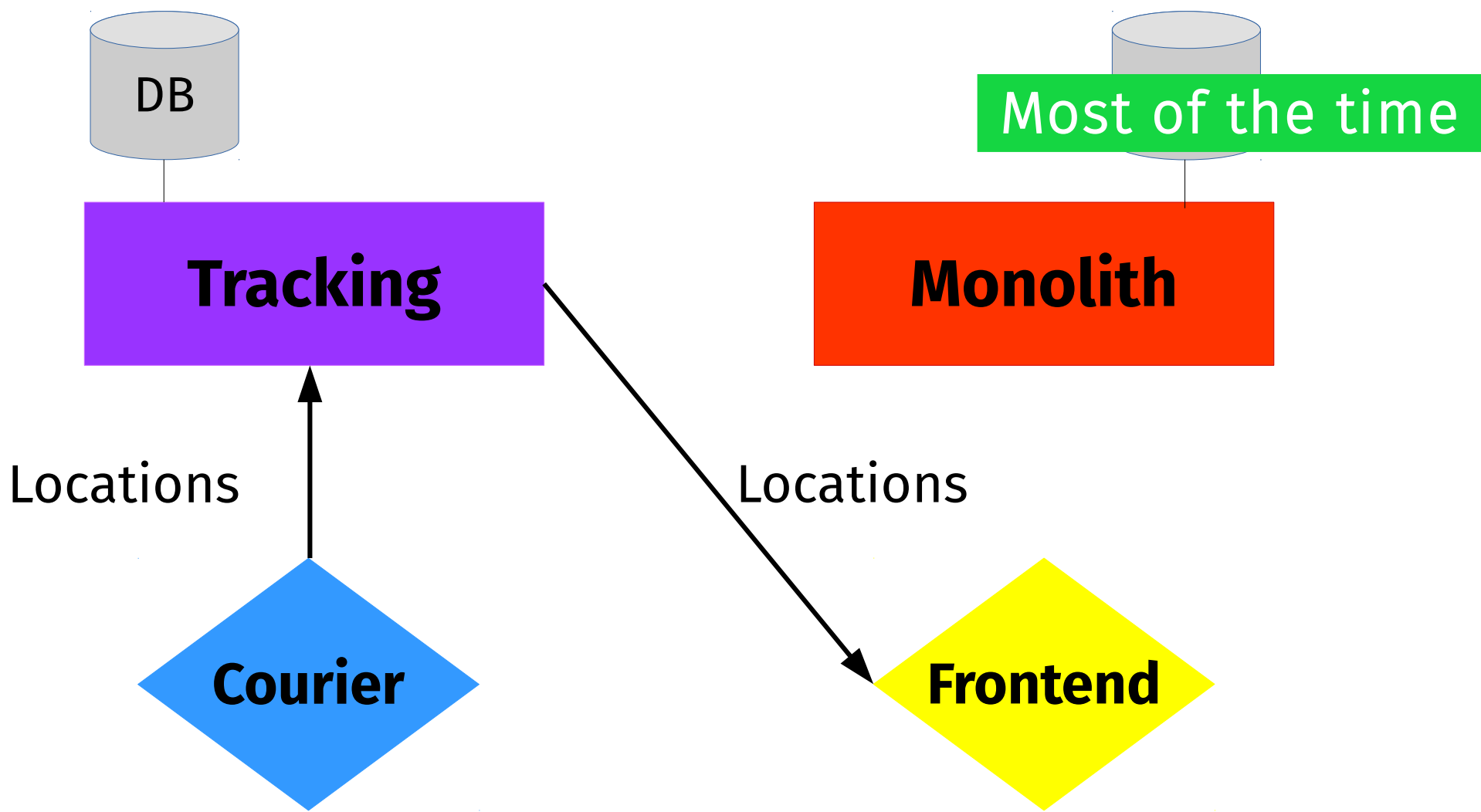
```
def connect(%{"token" => token}, socket) do
  token
  |> JWT.verify_token
  |> respond_based_on_token_contents(socket)
end
```



```
defp login_user(%{"role" => role, ...}, socket) do
  {:ok, assign(socket, :user, %{role: role, ...})}
end
```

```
def join("courier:" <> courier_id, _, %  
        {assigns: %{user: user}}) do  
  if authorized?(courier_id, user) do  
    # login  
  else  
    # unauthorized  
  end  
end
```

```
defp authorized?("", _), do: false
defp authorized?(courier_id, %{courier_ids: ids}) do
  Enum.member?(ids, String.to_integer(courier_id))
end
defp authorized?(_, _), do: false
```





Fulfillment

2 aisle
pasillo

STANDARD
↓

1

45



Fulfillment

CRUD

2 aisle
pasillo



Fulfillment

CRUD

Experience

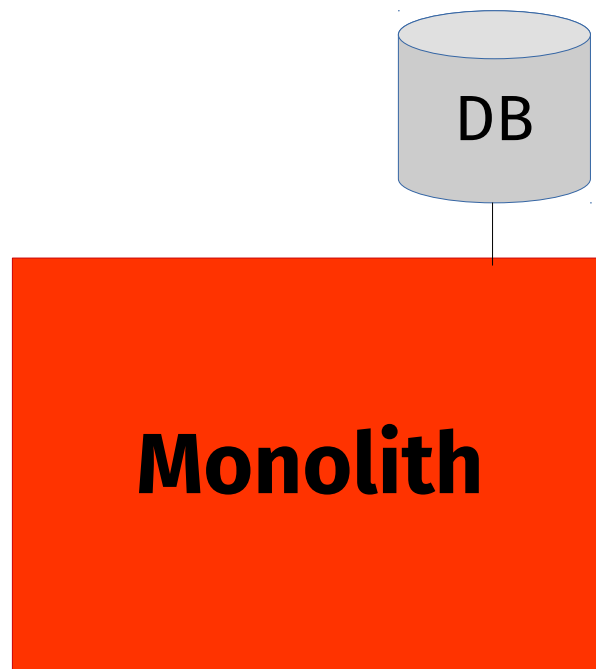
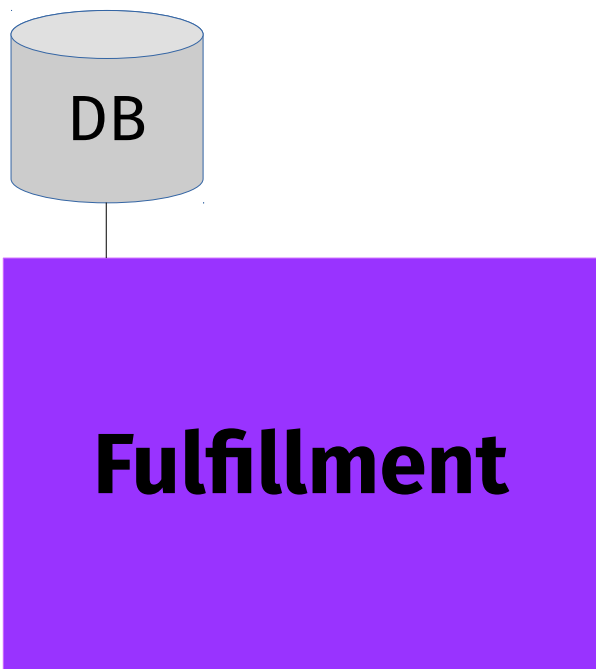
2 aisle
pasillo

STANDARD
↓

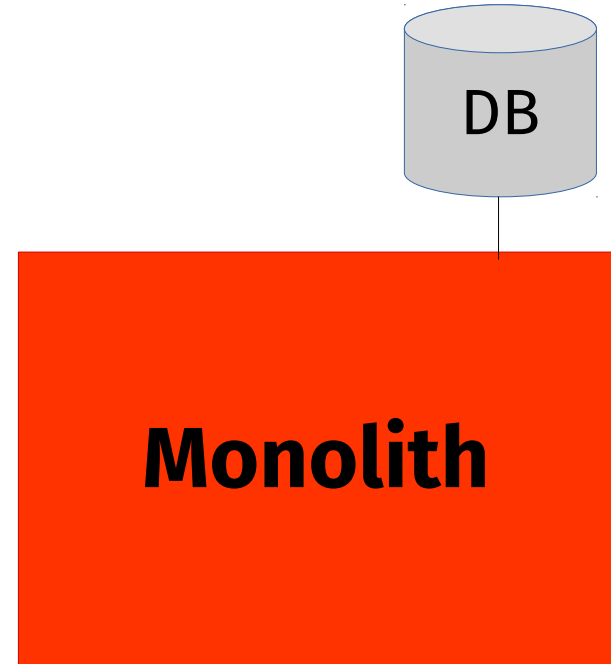
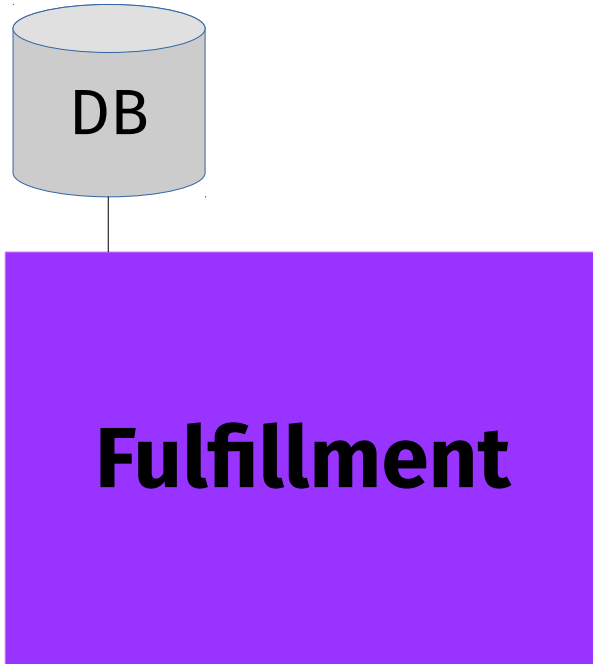
1

Picking with Delivery
DMS

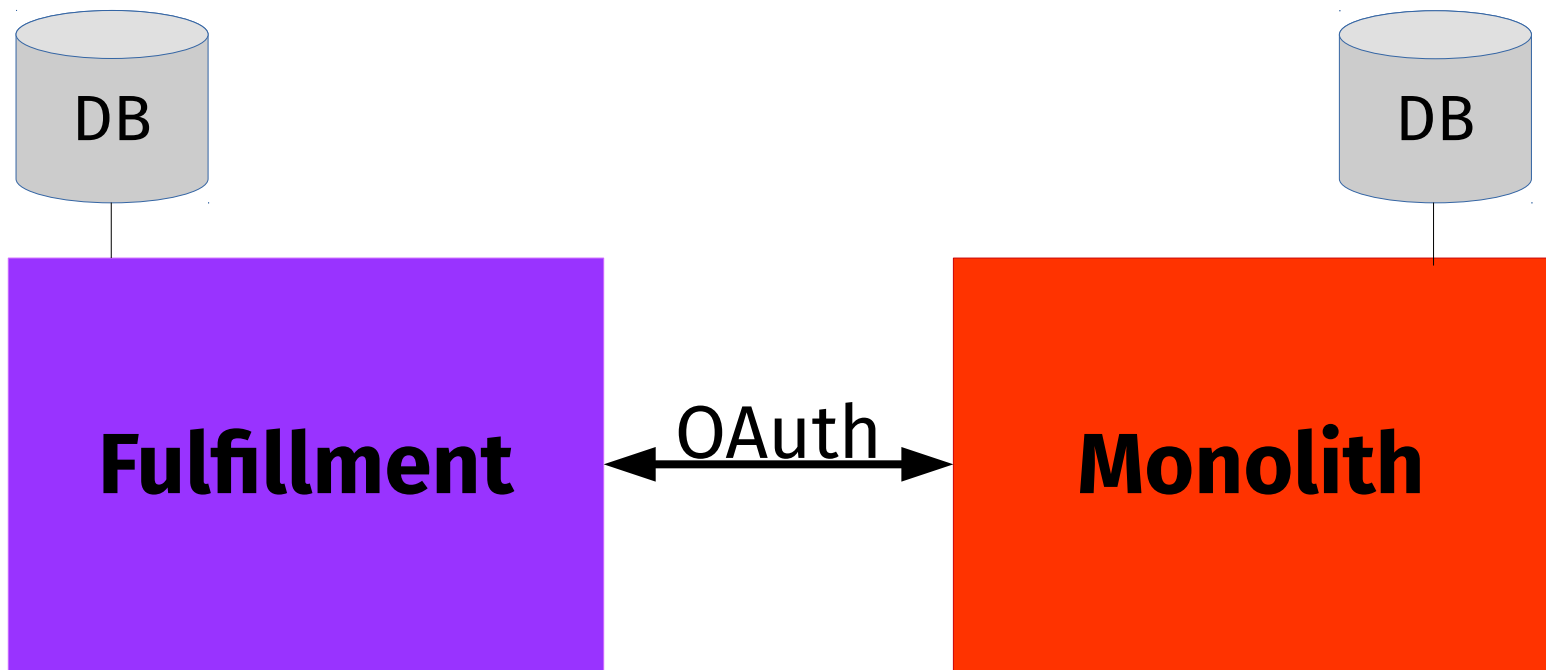
45
Pallets



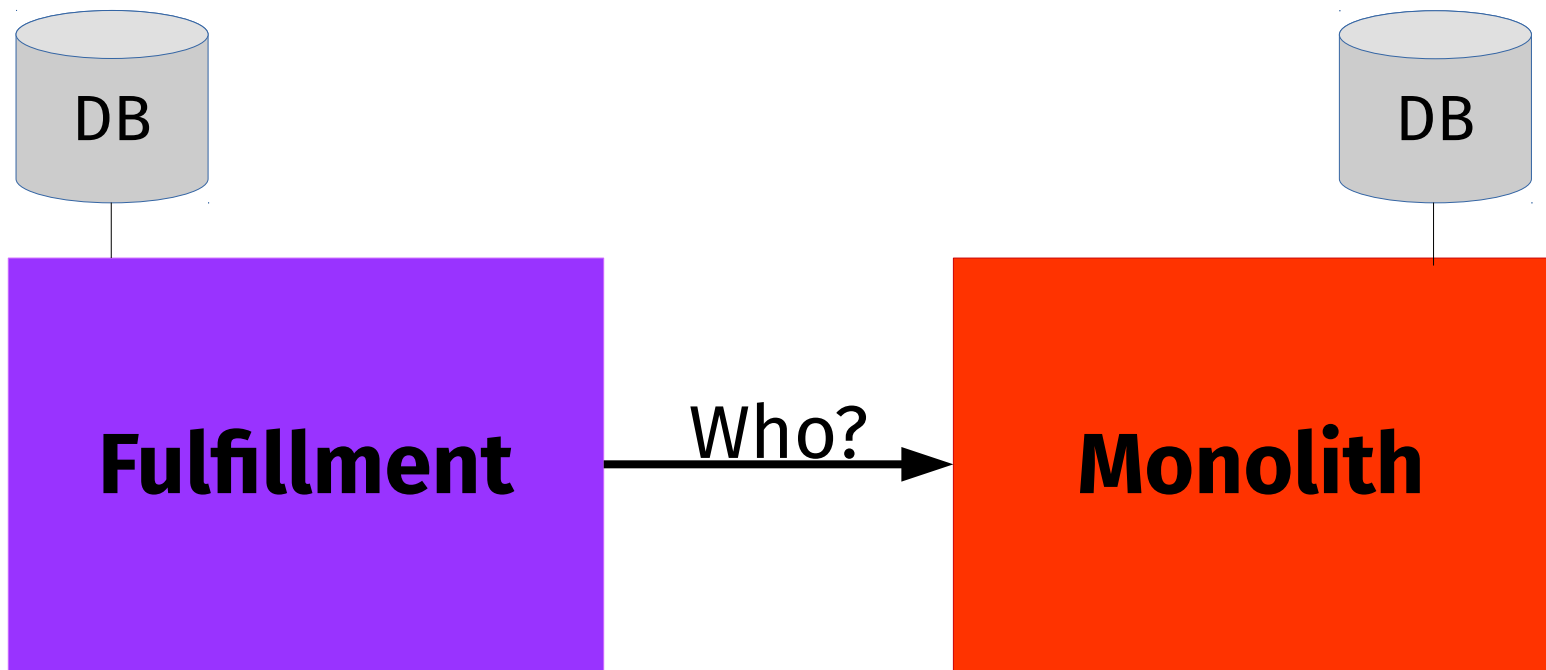
Own UI



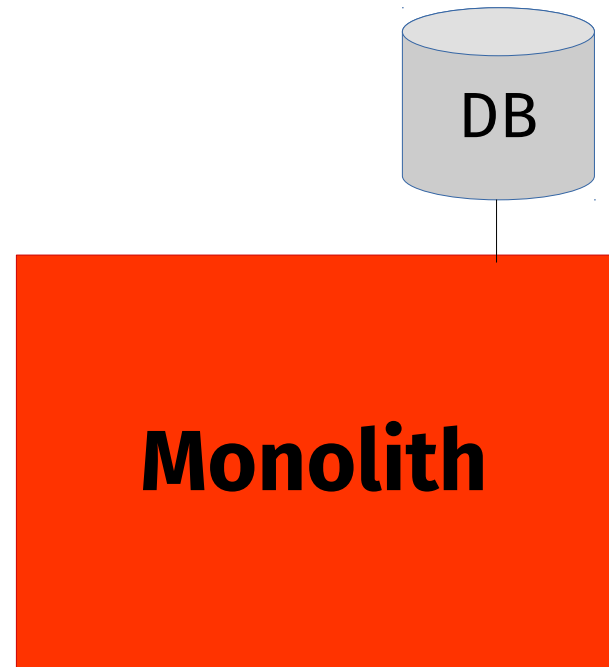
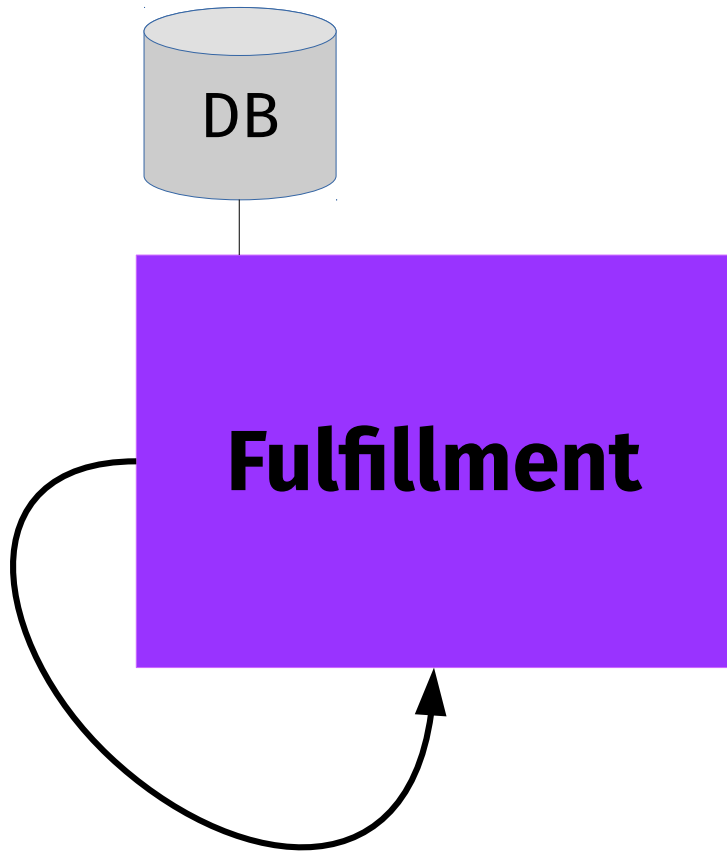
OAuth



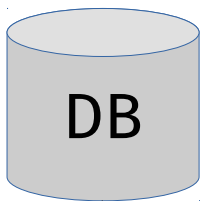
`/current_user`



Products



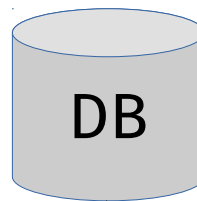
Order



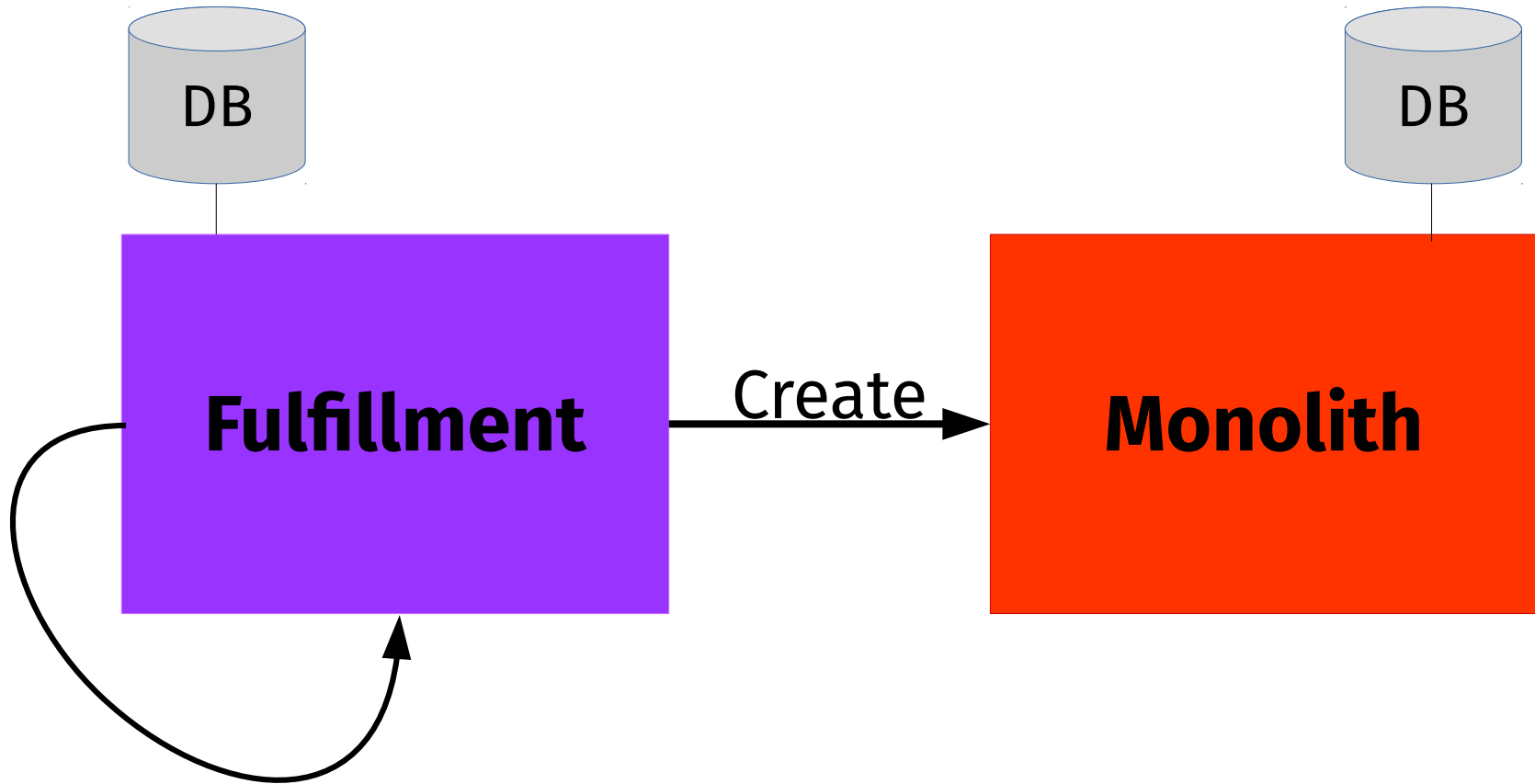
Fulfillment

Valid?

Monolith



Packaging



Rail-way oriented
programming!

Railway Oriented Programming



```
defmodule Fulfillment.OrderCreation do
  def create(params, customer, auth) do
    with {:ok, order} <- validate_order(params, customer),
         {:ok, order} <- validate_in_backend(order, auth),
         {:ok, order} <- Repo.insert(order) do
      {:ok, order}
    else
      {:error, changeset} -> {:error, changeset}
    end
  end
end
```

Validate internally

```
defmodule Fulfillment.OrderCreation do
  def create(params, customer, auth) do
    with {:ok, order} <- validate_order(params, customer),
         {:ok, order} <- validate_in_backend(order, auth),
         {:ok, order} <- Repo.insert(order) do
      {:ok, order}
    else
      {:error, changeset} -> {:error, changeset}
    end
  end
end
```


Validate externally

```
defmodule Fulfillment.OrderCreation do
  def create(params, customer, auth) do
    with {:ok, order} <- validate_order(params, customer),
         {:ok, order} <- validate_in_backend(order, auth),
         {:ok, order} <- Repo.insert(order) do
      {:ok, order}
    else
      {:error, changeset} -> {:error, changeset}
    end
  end
end
```

Save

```
defmodule Fulfillment.OrderCreation do
  def create(params, customer, auth) do
    with {:ok, order} <- validate_order(params, customer),
         {:ok, order} <- validate_in_backend(order, auth),
         {:ok, order} <- Repo.insert(order) do
      {:ok, order}
    else
      {:error, changeset} -> {:error, changeset}
    end
  end
end
```

Done

```
defmodule Fulfillment.OrderCreation do
  def create(params, customer, auth) do
    with {:ok, order} <- validate_order(params, customer),
         {:ok, order} <- validate_in_backend(order, auth),
         {:ok, order} <- Repo.insert(order) do
      {:ok, order}
    else
      {:error, changeset} -> {:error, changeset}
    end
  end
end
```


Error

```
defmodule Fulfillment.OrderCreation do
  def create(params, customer, auth) do
    with {:ok, order} <- validate_order(params, customer),
         {:ok, order} <- validate_in_backend(order, auth),
         {:ok, order} <- Repo.insert(order) do
      {:ok, order}
    else
      {:error, changeset} -> {:error, changeset}
    end
  end
end
```

No Kafka / Message Queue?

Be careful with new
technologies!

Microservices!

~~Microservices!~~

"Macroapplications"

All challenges are
clearly technical



Stakeholders



Your team



First Major Version



Get the others on board

Workshops



To know it,
you got to build it

KNOW THE
RULES

Pair Adopter with Alchemist

KNOW THE
RULES

Pair Adopter with Alchemist

Alchemist doesn't touch keyboard

KNOW THE
RULES

The
Pragmatic
Programmers

Your Elixir Source

Knowledge

Programming Phoenix

Productive |> Reliable |> Fast



Chris McCord,
Bruce Tate,
and José Valim

edited by Jacquelyn Carter

Adopting Elixir

From Concept to Production



Ben Marx, José Valim, Bruce Tate

edited by Jacquelyn Carter

*“We are still using this
like Rails – we should
use GenServers!”*

Someone on your team

You don't need to kill
your Monolith,
Complement it

Your Apps



Enjoy using Elixir effectively to help your company

Tobias Pfeiffer

@PragTob
pragtob.info



L I E F E R Y